

TEMA 03 • ESTRATEGIA DEL DATO, ML & DL

Fundamentos de *deep* *learning*.

Redes neuronales explicadas con paellas, memes y mucho café. De la neurona a las arquitecturas que mueven el mundo.

ASIGNATURA	Estrategia del Dato, ML y DL
TEMA	03 • Fundamentos de Deep Learning
DURACIÓN	120 minutos
SESIÓN	03 / 07

LO QUE VEREMOS HOY

Dos bloques. Una clase. De la neurona a las grandes arquitecturas.

BLOQUE I · 60 MIN · LA COCINA POR DENTRO

- ¿Qué es el *deep learning*? La paella inteligente
- DL vs ML: cuándo usar cada uno
- La neurona artificial: 4 piezas
- Capas: cómo se organiza una red
- Cómo aprende: forward, error, backprop
- Descenso del gradiente: *abuelita vs Hulk*

BLOQUE II · 60 MIN · LAS ARQUITECTURAS ESTRELLA

- Funciones de activación: el grifo de la neurona
- *CNN*: el ojo entrenado (Instagram, radiografías)
- *RNN / LSTM*: la red con memoria (Spotify, Whatsapp)
- *GAN*: el falsificador y el policía (deepfakes)
- Quiz interactivo: ¿qué arquitectura usarías?

La cocina por *dentro*.

Qué es una red neuronal, de qué piezas se compone y cómo aprende a hacer su trabajo.

¿Qué es el *deep learning*?

Una rama del **machine learning** que usa *redes neuronales con muchas capas* para aprender directamente de los datos. Cuantas más capas, más «profunda» — de ahí *deep*.

ANALOGÍA · LA PAELLA

Programación clásica: tienes la receta escrita paso a paso. Si pones azafrán de más, salada queda salada.

Machine learning: pruebas mil paellas, te dicen «esta sí, esta no» y aprendes una regla.

Deep learning: pruebas un millón de paellas y la red aprende sola que importan el agua, el sofrito, el caldo... cosas que ni tú habías pensado.

PROGRAMACIÓN CLÁSICA

Datos + reglas → resultado

El humano escribe las reglas. «*Si lleva azafrán, es paella valenciana.*»

MACHINE LEARNING

Datos + resultados → reglas

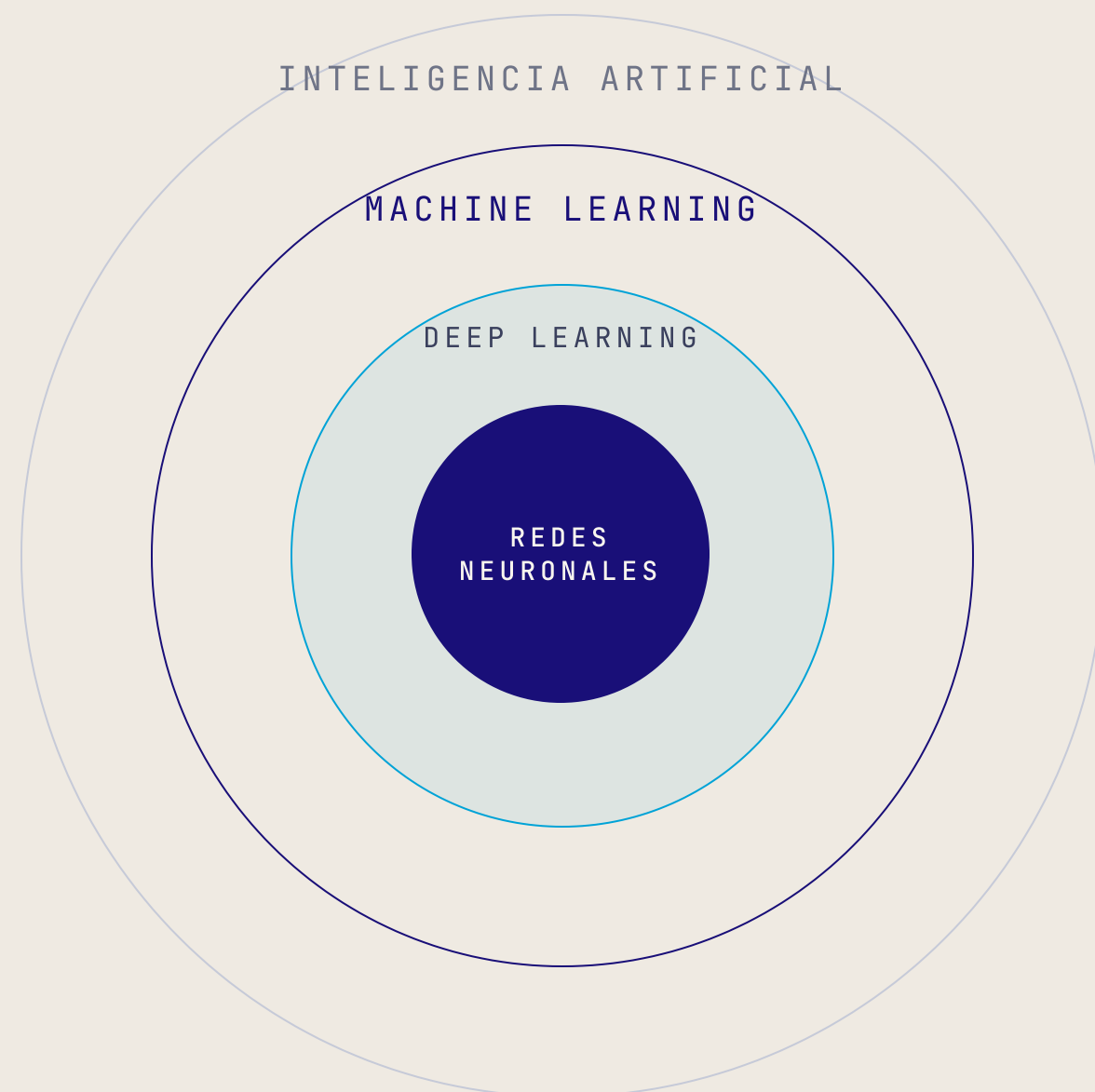
El modelo descubre las reglas. Pero tú le dices qué mirar (ingredientes, peso...).

DEEP LEARNING

Datos crudos → todo

Le pasas la *foto* de la paella y la red descubre sola qué importa. Sin recetas, sin pistas.

IA, ML, DL: *matrioskas*.



INTELIGENCIA ARTIFICIAL

«Que la máquina haga cosas inteligentes.» Caben Siri, los semáforos inteligentes y un sistema experto de los 80.

MACHINE LEARNING

IA que aprende de datos. *Spotify recomendándote música* con regresión logística.

DEEP LEARNING

ML con redes neuronales profundas. *El sistema que reconoce tu cara* en Instagram para ponerte el filtro de perro.

¿Y entonces, *cuál uso*?

Regla mental: si tienes **pocos datos tabulares** (un Excel), ML clásico. Si tienes **muchas imágenes, audios o textos** crudos, deep learning.

	MACHINE LEARNING	DEEP LEARNING
DATOS QUE NECESITA	Funciona con cientos o miles de filas. Un Excel decente.	Quiere <i>millones</i> . ImageNet tiene 14 millones de fotos.
TÚ HACES...	Ingeniería manual: «mira el peso, la edad, el código postal».	Le pasas el dato crudo y la red descubre <i>sola</i> qué mirar.
¿POR QUÉ DECIDIÓ ESO?	Te lo puede explicar: árboles, coeficientes...	<i>Caja negra</i> . Acierta mucho pero no te lo cuenta.
HARDWARE	Tu portátil de ofertas vale.	<i>GPU</i> sí o sí. La factura de la luz se nota.
CASO TÍPICO	Predecir si un cliente se da de baja del gimnasio.	Detectar gatos en fotos de Instagram.

Una *neurona* es un cocinero.

Recibe ingredientes (entradas), decide cuánto importa cada uno (pesos), añade su toque personal (sesgo) y al final aplica una decisión: ¿lo saco o no? (activación).

01 · ENTRADAS

Los ingredientes

Datos que llegan: píxeles, palabras, números... $x_1, x_2, x_3...$

Ej. cantidad de azafrán, agua, arroz.

02 · PESOS

Cuánto importa cada uno

El cocinero aprende que el azafrán es crítico (peso alto) y el agua, regulara. $w_1, w_2, w_3...$

03 · SESGO (BIAS)

El toque personal

Constante que se suma siempre. La «mano del cocinero» — ese empujón base independiente de los ingredientes. b

04 · ACTIVACIÓN

¿Lo saco o no?

Función no lineal que decide la salida final. «*Está lista la paella*» o «*sigue cocinando*».

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

salida = $f(z)$

TRADUCCIÓN HUMANA

Cada ingrediente se multiplica por su importancia, se suman todos, se añade el toque del cocinero, y la activación decide qué sale por la puerta.

Una *neurona viva*.

¿Es una buena paella? Mueve los pesos y el sesgo. Cuando la salida supera **0.5** la neurona dice «*sí, paella aprobada*».

PESOS DEL COCINERO

Azafrán ($x_1=0.8$)		1.5
Arroz ($x_2=1.0$)		0.8
Pulpo* ($x_3=0.6$)		-1.2
Sesgo (b)		-0.5

* el pulpo en la paella va con peso negativo, sí o sí.

CÁLCULO EN VIVO

$z = 0.78$
 $\sigma(z) = 0.69$



¡Paella aprobada!

Ese **0.5** es la frontera de decisión. Por debajo: «no». Por encima: «sí». La activación que estás viendo es la **sigmoide**, que aplasta cualquier número entre 0 y 1.

Una neurona sola es un becario. Muchas, son *una empresa*.

Conectas neuronas en columnas (capas). Cada capa aprende algo más abstracto que la anterior. Si la red es muy profunda, decimos que es *deep*.

ENTRADA · EL BECARIO

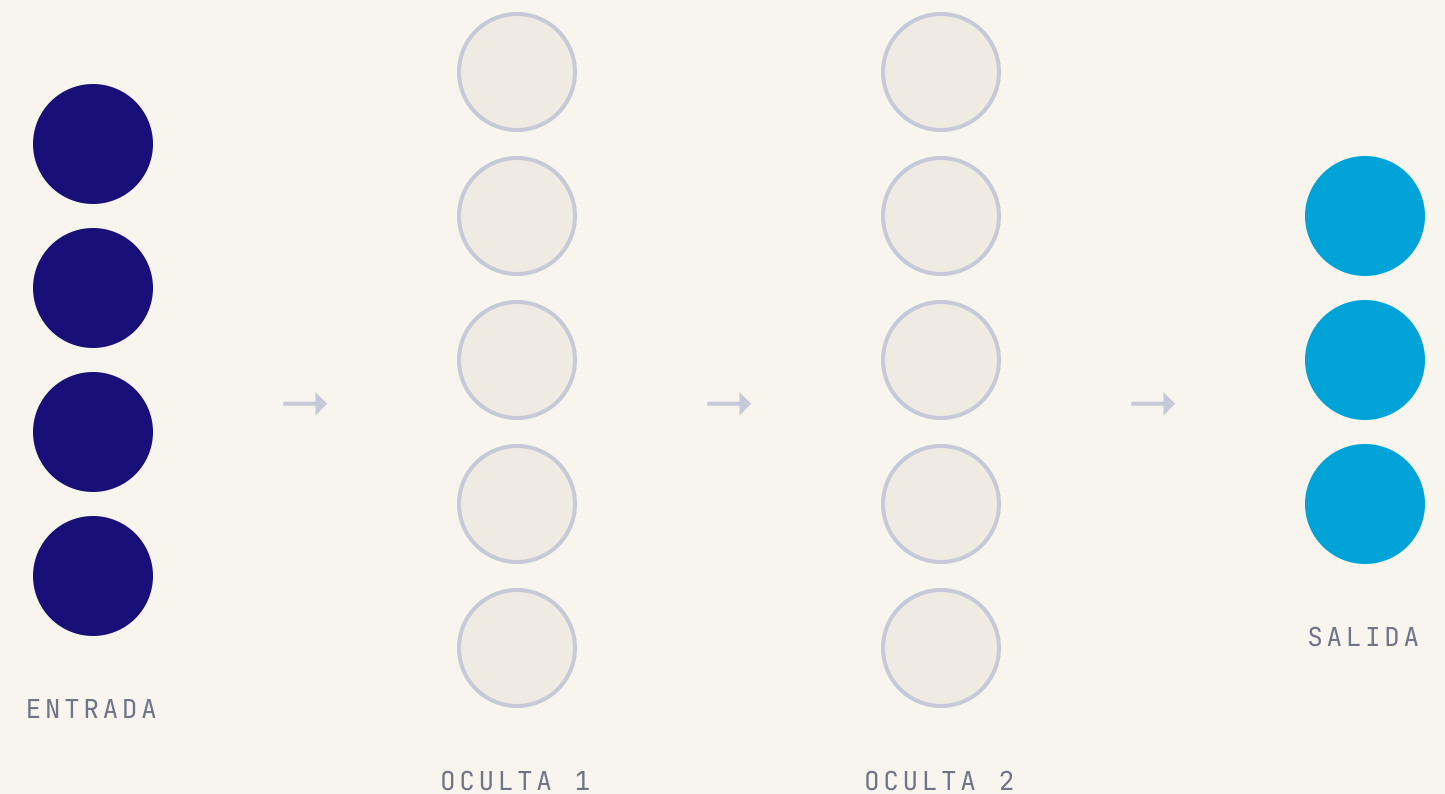
Recibe el dato crudo. Una neurona por píxel, por palabra, por columna del Excel.

OCULTAS · LOS MANDOS INTERMEDIOS

Procesan, resumen, abstraen. *«Esto que viene parece una oreja... esto parece pelo... ¡esto es un perro!».*

SALIDA · EL DIRECTOR

Toma la decisión final. Probabilidades para clasificar, un número para regresión.



Capa 1: detecta bordes. Capa 4: detecta orejas. Capa 7: *«es un golden retriever».*

Cómo *aprende* una red.

Como un niño aprendiendo a montar en bici: prueba → se cae → ajusta el equilibrio → vuelve a probar. Mil veces. Hasta que sale solo.

01 Forward · «Hago mi predicción»

El dato cruza la red. Sale un número. *«Yo digo que esto es un gato.»*

02 Loss · «¿Cuánto la he cagado?»

Comparas con la respuesta correcta. *Era un perro.* La función de pérdida pone número al fallo.

03 Backprop · «Reparto culpas»

El error viaja hacia atrás. Cada peso recibe su parte: *«oye, w_3 , tú me has llevado por mal camino».*

04 Gradiente · «Ajusto un poquito»

Cada peso se mueve un paso pequeño en la dirección que reduciría el error.

05 Repite · 10.000 veces

Una pasada por todos los datos = una *epoch*. Hacen falta muchas para que la red converja.

ANALOGÍA · EL NIÑO Y LA BICI

1. **Pedalea** y se desequilibra (forward).
2. **Se cae** a la izquierda (loss).
3. **Su cerebro nota** que ha cargado mucho peso a la izquierda (backprop).
4. **La próxima vez** compensa un poquito a la derecha (gradiente).
5. **Otra vez.** Y otra. Y otra.

CUIDADO

Si el ajuste es muy bestia se cae para el otro lado.

Si es muy pequeño, no aprende nunca. Eso lo veremos en la siguiente slide.

Descenso del gradiente: *abuelita vs Hulk*.

El learning rate (η) es el **tamaño del paso**. Imagina que estás en una colina con los ojos vendados y quieres llegar al valle.
¿A dónde llegas con pasitos de abuelita? ¿Y con saltos de Hulk?

TU PASO POR EL MUNDO

Learning rate (η) 0.10

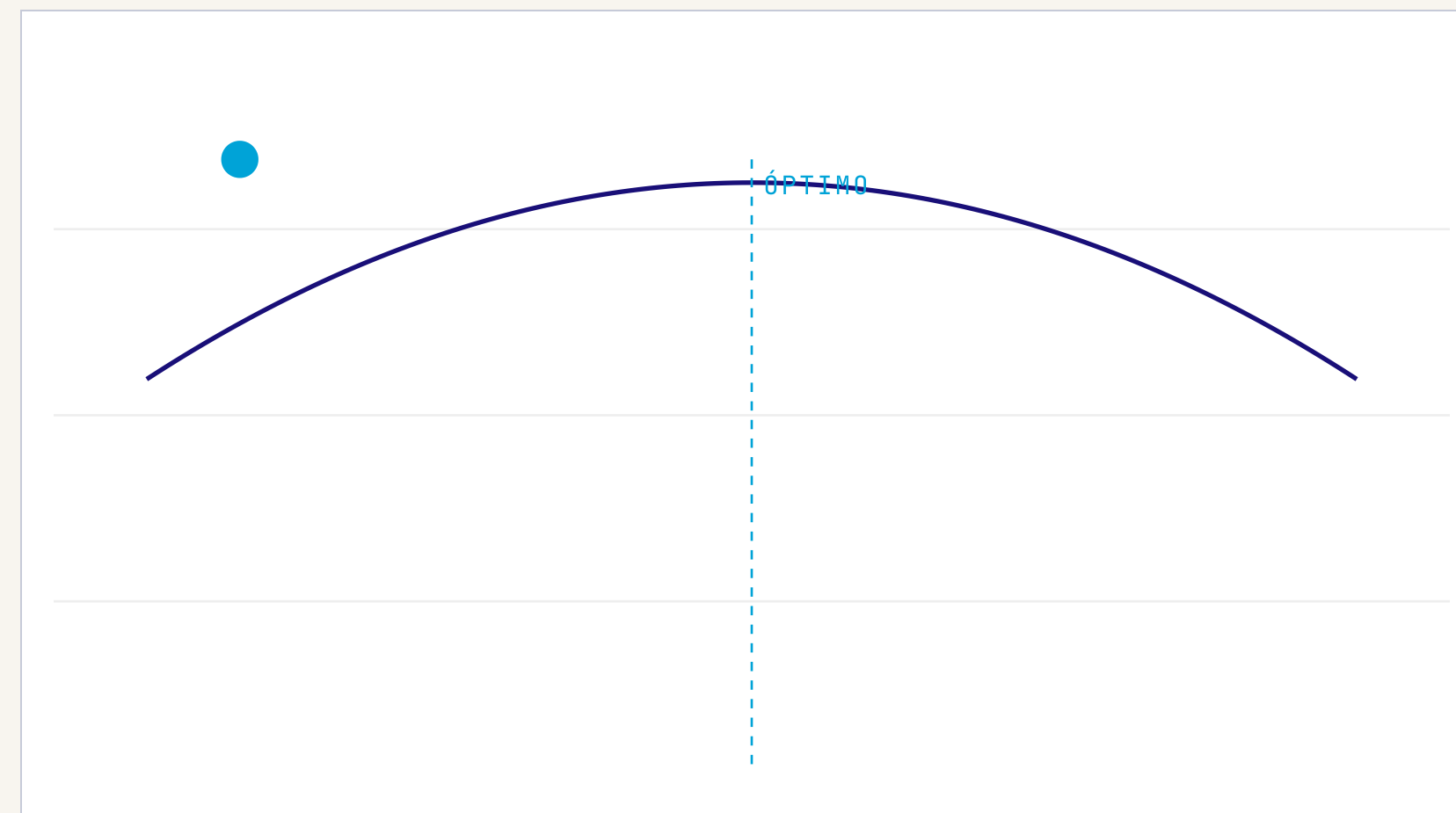
🙄 ABUELITA

😊 BIEN

♥️ HULK

► LANZAR LA PELOTA

Pulsa «Lanzar la pelota» para ver qué pasa.



🙄 ABUELITA · H MUY BAJO

Pasitos diminutos. Llega al valle... cuando ya no hay clase.

😊 BIEN · H EQUILIBRADO

Pasos razonables. Baja, frena cerca del valle, encuentra el punto.

♥️ HULK · H MUY ALTO

Saltos enormes. «SMASH!» Pasa por encima del valle. No converge.

II PARA. RESPIRA. RESUME.

¿Te llevas *esto* de aquí?

1 Una neurona = ingredientes $\times$ pesos + sesgo, pasado por una activación.
Si esto suena, vamos bien.

2 Una red profunda = muchas capas de neuronas que se pasan información.
Cada capa aprende algo más abstracto.

3 Aprende repitiendo: predigo $\rightarrow$ mido el error $\rightarrow$ reparto culpas $\rightarrow$ ajusto pesos.
Forward, loss, backprop, gradiente.

4 El learning rate es el tamaño del paso.
Abuelita lento, Hulk loco, equilibrio justo.

Las arquitecturas *estrella.*

Activaciones • CNN • RNN/LSTM • GAN. Las
cuatro herramientas que mueven la IA moderna.

La activación es *el grifo*.

Cada neurona calcula un número (z). La activación decide cómo «sale el agua»: a chorro, gota a gota o nada. **Sin activaciones no lineales, una red de 100 capas equivale a una sola.** Por eso son críticas.

SIGMOIDE · Σ

El portero educado

Aplasta cualquier número entre 0 y 1. *«0.87 de probabilidad de que sea gato.»*

Uso: salida binaria. **Problema:** en capas ocultas, mata el gradiente.

RELU

El simplón eficaz

«Si es positivo, pásalo. Si es negativo, cero.» *$\max(0, z)$.*

Uso: **capas ocultas, por defecto.** Rápida y resuelve el gradiente.

TANH

La sigmoide centrada

Como sigmoide pero entre -1 y 1. Centrada en cero, mejor para capas ocultas.

Uso: RNN clásicas, modelos donde importa el signo.

LEAKY RELU

ReLU con propinilla

Como ReLU, pero los negativos no son 0 sino un poquito ($0.01 \cdot z$). Evita «neuronas muertas».

Uso: cuando ReLU «mata» neuronas en el entrenamiento.

SOFTMAX

El repartidor de tarta

Recibe varios números y los convierte en probabilidades que suman 1. *«40% gato, 35% perro, 25% hámster.»*

Uso: **salida multiclase.**

LINEAL

Sin filtro

No hace nada: $f(z)=z$. Devuelve el número tal cual.

Uso: **salida de regresión** (predecir un precio, una temperatura).

Las activaciones en *vivo*.

ELIGE FUNCIÓN

SIGMOIDE

RELU

TANH

L.RELU

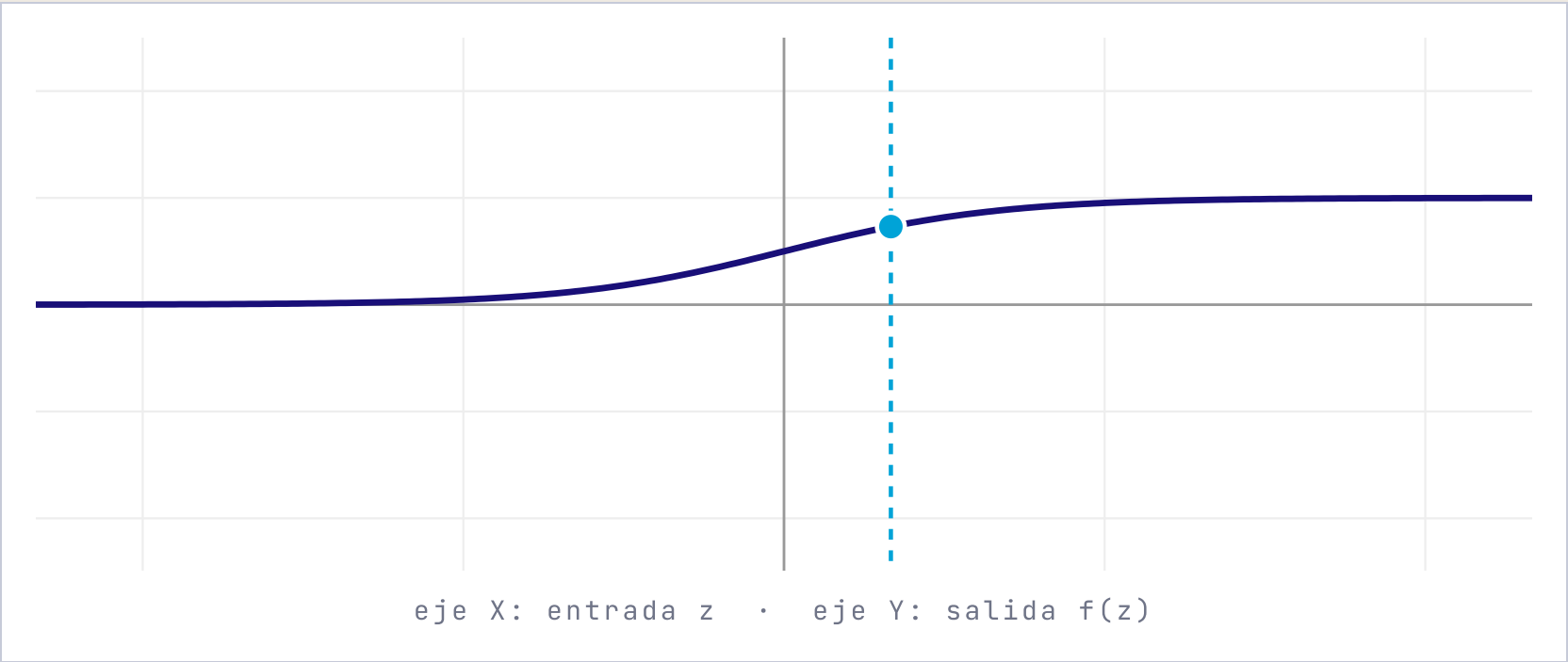
MUEVE LA ENTRADA Z

z

1.0

f(z) = 0.731

Sigmoide: aplasta z entre 0 y 1. Ideal para probabilidad binaria.



Si dudas, esta *chuleta*.

CAPAS OCULTAS

ReLU. Punto.

Es rápida, evita el gradiente desvaneciente y casi siempre funciona. Si ves «neuronas muertas», prueba Leaky ReLU.

SALIDA · CLASIFICACIÓN BINARIA

Sigmoide

Una sola probabilidad: ¿es spam o no? Combínala con *binary cross-entropy*.

SALIDA · CLASIFICACIÓN MULTICLASE

Softmax

«¿Es perro, gato o hámster?» Combínala con *categorical cross-entropy*.

SALIDA · REGRESIÓN

Lineal (o sin activación)

«¿Cuánto vale este piso?» Quieres un número, no una probabilidad. Usa *MSE* como loss.

LO QUE TODA ACTIVACIÓN DEBE CUMPLIR

- **No lineal** en ocultas — si no, la red se vuelve tonta
- **Diferenciable** — backprop la necesita
- Que *no mate el gradiente*

CNN: el *ojo* entrenado.

El filtro de Instagram, las radiografías, los coches autónomos, el reconocimiento facial. Todo lo «visual» en IA hoy es CNN o un descendiente.

CNN

Una foto es una *matriz de números*.

Para un ordenador no hay «bigotes de gato». Hay una matriz de números entre 0 (negro) y 255 (blanco). Una CNN aprende a reconocer patrones en esa matriz aplicando *filtros* que se deslizan por toda la imagen.

ANALOGÍA · BUSCANDO A WALLY CON UNA LUPA

Tienes una lámina enorme y una lupita. **La pasas por toda la página**, mirando trozos pequeños. Cuando coincide con «camiseta de rayas rojas y blancas», anotas: «aquí hay algo».

Eso es una *convolución*: una lupita (kernel) que recorre la imagen buscando un patrón.

Y LO MEJOR

No le dices qué buscar. La red *aprende sola* qué lupas son útiles. En las primeras capas inventa lupas para detectar bordes; en las últimas, para detectar caras enteras.

IMAGEN 5×5 (UN DÍGITO 1 MUY PIXELADO)

0	0	9	0	0
0	0	9	0	0
0	0	9	0	0
0	0	9	0	0
0	0	9	0	0



FILTRO «LÍNEA VERTICAL»

-1	+2	-1
-1	+2	-1
-1	+2	-1

Este kernel se «enciende» cuando la imagen tiene *una línea vertical*.

Pasa *la lupa* por la imagen.

Pulsa «**Siguiente**» para mover el kernel un paso. Cada celda del mapa de features se ilumina con el resultado del producto escalar entre el kernel y la región actual.

IMAGEN 5×5

0	0	9	0	0
0	0	9	0	0
0	0	9	0	0
0	0	9	0	0
0	0	9	0	0

SIGUIENTE PASO ►

RESET

Posición: (1, 1) — pulsa «Siguiente».

KERNEL 3×3 (VERTICAL)

-1	+2	-1
-1	+2	-1
-1	+2	-1

Detecta líneas verticales: pesos altos en el centro, negativos a los lados.

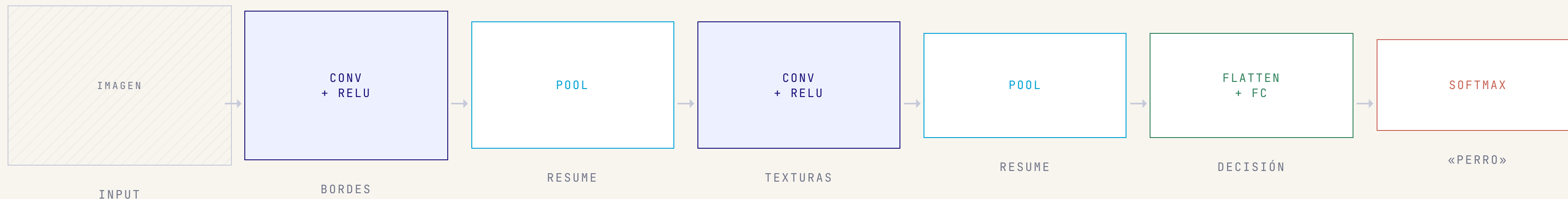
MAPA DE FEATURES 3×3

.	.	.
.	.	.
.	.	.

Cuanto más oscuro, más activado el filtro en esa región.

De la foto a «*es un perro*».

Una CNN encadena **convoluciones** (lupas que detectan patrones) con **poolings** (zooms hacia atrás que resumen). Al final, una capa fully-connected y un softmax dictan sentencia.



CÓMO CRECE LA ABSTRACCIÓN

Capa 1: «hay una raya inclinada». **Capa 3:** «esto parece una oreja». **Capa 7:** «cara de golden retriever». La red va componiendo: bordes → texturas → partes → objetos.

LENET · 1998

Yann LeCun. La CNN abuela. Lee dígitos escritos a mano.

ALEXNET · 2012

Ganó ImageNet por goleada. Encendió toda la era moderna del DL.

VGG · 2014

Apila muchísimas capas pequeñas (3×3). Simple y profunda.

RESNET · 2015

Conexiones residuales: «si esto no ayuda, salta y déjalo igual».

II RESUMEN CNN

Si te quedas con *tres frases*:

1

Una imagen es una matriz de números.

Para la red no hay «pelo», hay píxeles.

2

Un kernel es una lupita que detecta un patrón.

La red aprende qué lupas le sirven.

3

Las primeras capas ven bordes; las últimas, objetos.

Abstracción jerárquica.

RNN: la red con *memoria*.

Para todo lo que va en orden: texto, voz, música, series temporales. La predictiva del WhatsApp y el «autoplay» de Spotify.

RNN

3.4 · ¿POR QUÉ HACE FALTA?

«No *como* patatas» $\neq$ «*Como* no patatas».

Las CNN ven un trozo a la vez sin recordar nada. Para entender una frase necesitas **arrastrar contexto**: lo que dijiste hace 3 palabras importa para entender la cuarta.

ANALOGÍA · WHATSAPP PREDICTIVO

Escribes «*vamos a por una*» y el teclado te sugiere «**caña**», no «**hipoteca**». Para acertar tiene que recordar las palabras anteriores.

Eso es una RNN: una red que *arrastra un estado* de paso en paso.

PROBLEMA · MEMORIA DE PEZ

La RNN simple olvida rápido.

Si la frase es larga, lo que dijiste al principio se diluye. La red recuerda bien las últimas 3-4 palabras, las anteriores... se evaporan.

SOLUCIÓN · LSTM Y GRU

Memoria con compuertas.

Tres «llaves» que deciden qué entra, qué sale y qué se conserva. Por fin podemos recordar 100 palabras atrás.

$$h_t = f(h_{t-1}, x_t)$$

$$\hat{y}_t = g(h_t)$$

TRADUCCIÓN

«Mi memoria de ahora (h_t) depende de mi memoria de antes (h_{t-1}) y de lo que veo ahora (x_t).»

La salida de cada paso sale de esa memoria.

Una RNN *en acción*.

Pulsa «Siguiente palabra» y observa cómo el *estado oculto* (las barritas) cambia con cada token. Cada barra es una neurona de la memoria.

FRASE DE ENTRADA

Vamos

a

por

una

SIGUIENTE PALABRA ►

RESET

ESTADO OCULTO H_T (MEMORIA)

Estado inicial: vacío. La red aún no ha visto nada.

PREDICCIÓN DE LA SIGUIENTE PALABRA

— pulsa «Siguiente» —

A medida que avanzas, la memoria acumula contexto y las predicciones tienen más sentido.

Memoria con *tres llaves*.

Una LSTM (Long Short-Term Memory) es una RNN con compuertas que deciden qué información olvidar, qué guardar y qué sacar.

ANALOGÍA · LA LIBRETA DE LA ABUELA

Imagina la libreta de tu abuela, esa de las recetas. Tres acciones:

 **Olvidar:** tachas la receta del bizcocho que nunca sale (forget gate).

 **Guardar:** apuntas la receta nueva del cocido (input gate).

 **Consultar:** abres la libreta para sacar lo que necesitas (output gate).

La LSTM hace eso en cada paso de la secuencia, automáticamente.

LSTM · HOCHREITER & SCHMIDHUBER, 1997

Long Short-Term Memory

Tres compuertas (entrada, olvido, salida) y una *línea de estado* que viaja a lo largo de toda la secuencia. Resuelve el gradiente desvaneciente.

GRU · CHO ET AL., 2014

Gated Recurrent Unit

LSTM simplificada: sólo dos compuertas. Menos parámetros, entrena más rápido y rinde casi igual.

¿DÓNDE LAS USAS A DIARIO?

Spotify (siguiente canción), **Google Translate** (pre-Transformers), **Alexa/Siri** y **series temporales** (bolsa, demanda eléctrica).

GAN: el *falsificador* y el *policía*.

Dos redes que compiten entre sí. El resultado:
deepfakes, caras de gente que no existe,
arte sintético.

GAN

Dos redes *peleándose*. En serio.

Goodfellow, 2014. Idea genial: en vez de pedirle a una red que «genere un cuadro de Picasso», pones **a dos redes a competir**. Una intenta engañar; la otra intenta no ser engañada.

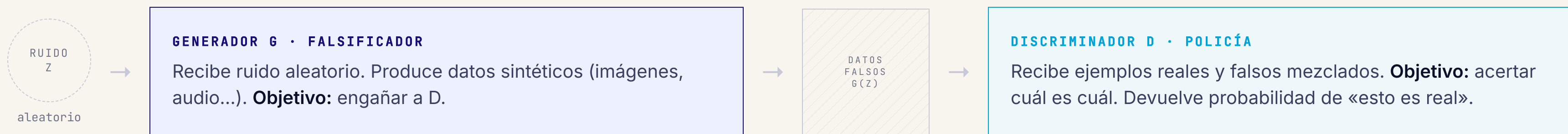
ANALOGÍA · FALSIFICADOR VS POLICÍA

Falsificador (Generador): empieza haciendo billetes patéticos. Se los enseña al policía.

Policía (Discriminador): los pilla todos al instante. *«Esto es una fotocopia, hombre.»*

Falsificador: mejora. Hace billetes algo mejores. El policía sigue pillándolos. **Falsificador:** mejora más. **Policía:** mejora también.

Tras miles de rondas, los billetes son *indistinguibles* de los reales. La cara que ves al final no existe.

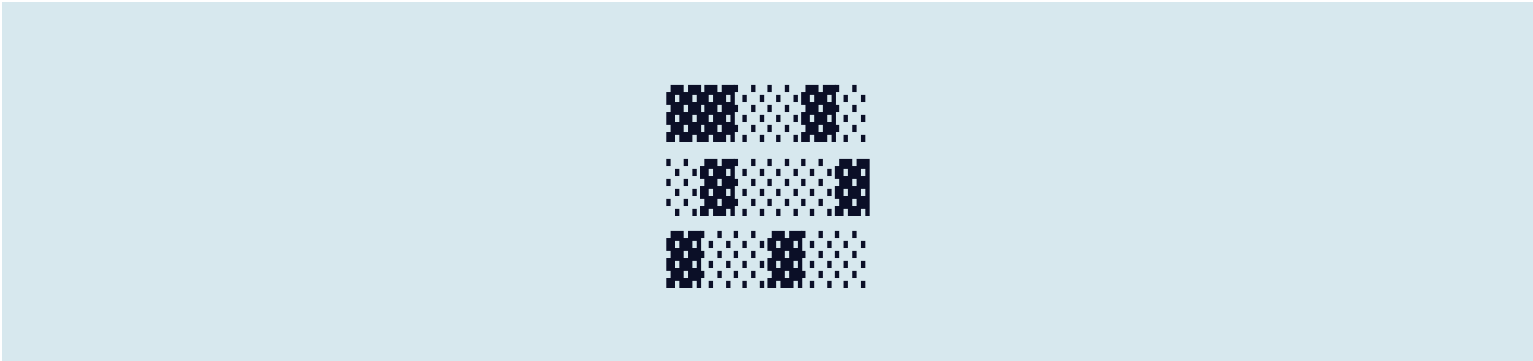


Cuando G y D llegan al equilibrio (Nash), el policía no puede distinguir → los billetes pasan por reales.

El falsificador y el policía, *turno a turno*.

Pulsa «Siguiente ronda». Verás cómo el falsificador mejora sus billetes mientras el policía sube su nivel.

 FALSIFICADOR (G)



Calidad

10%

 POLICÍA (D)

«¡Falso! Eso no engaña a nadie.»

Tasa de aciertos

95%

GAN en *el mundo real*.

CARAS QUE NO EXISTEN

thispersondoesnotexist.com

Cada vez que recargas, un humano nuevo. Generado por StyleGAN. Ninguno es real.

DEEPFAKES

Vídeos falsos hiperrealistas

El uso polémico. Cara de una persona en el cuerpo de otra. Implicaciones éticas y legales enormes.

ESTILO · CYCLEGAN

Caballos en cebras

Foto del caballo → mismo caballo pero rayado. Convierte fotos en cuadros de Monet, día en noche, etc.

DATA AUGMENTATION

Cuando no tienes datos reales

Genera radiografías sintéticas para entrenar otro modelo médico cuando los datos clínicos escasean.

SÚPER-RESOLUCIÓN

De foto borrosa a HD

El «zoom and enhance» de las pelis ahora es real. La GAN se inventa los detalles que faltan.

ARTE GENERATIVO

Cuadros, música, moda

Diseñadores la usan para explorar variantes. Christie's vendió un cuadro generativo por \$432.500.

¿Qué arquitectura para *qué problema*?

	CNN	RNN / LSTM	GAN
DATOS	Imágenes, vídeo, lo que tenga estructura espacial	Texto, voz, música, series temporales — orden importa	Cualquier tipo, sobre todo imágenes
¿QUÉ HACE?	<i>Reconocer</i> : clasifica, detecta, segmenta	<i>Predecir lo siguiente</i> : traducir, completar, prever	<i>Generar</i> : crea ejemplos sintéticos nuevos
ANALOGÍA	Pasar una lupa por la imagen	El WhatsApp predictivo con memoria	Falsificador vs policía
FAMOSAS	AlexNet, ResNet, EfficientNet, YOLO	LSTM, GRU, seq2seq	DCGAN, StyleGAN, CycleGAN

Es *tu turno*.

Para cada caso, decide: **CNN**, **RNN/LSTM** o **GAN**. Pulsa una opción y te diré si has acertado.

A • Detectar tumores en radiografías de tórax.

CNN

RNN / LSTM

GAN

B • Traductor automático español → inglés.

CNN

RNN / LSTM

GAN

C • Generar fotos de modelos para un catálogo de moda.

CNN

RNN / LSTM

GAN

D • Predecir el precio del Bitcoin de mañana a partir del histórico.

CNN

RNN / LSTM

GAN

E • Que el coche autónomo distinga peatones, semáforos y bicis.

CNN

RNN / LSTM

GAN

F • Convertir bocetos a mano alzada en imágenes fotorrealistas.

CNN

RNN / LSTM

GAN

Si *recuerdas* esto, has ganado.

DL

Deep learning = redes con muchas capas que aprenden de datos crudos.

La paella inteligente.

η

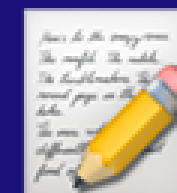
Forward $\rightarrow$ Loss $\rightarrow$ Backprop $\rightarrow$ Gradiente. Repite.

Y el η es abuelita $\leftrightarrow$ Hulk.



CNN para imágenes. La lupa que se desliza.

Bordes $\rightarrow$ texturas $\rightarrow$ objetos.



RNN/LSTM para secuencias. La memoria que se arrastra.

El WhatsApp predictivo.



GAN para generar. Falsificador vs policía.

Caras que no existen.



ReLU en ocultas.
Softmax/Sigmoide/Lineal en la salida.

La chuleta de activaciones.

¿Y los modelos tipo *ChatGPT*?

SESIÓN 04

Modelos de lenguaje y Transformers

Atención, BERT, GPT y los modelos fundacionales. Por qué sustituyeron a las RNN, qué hace que escalen tanto y cómo entrenarlos sin morir en el intento.

- **3Blue1Brown** · Serie en YouTube «Neural Networks». Las animaciones que cambian la vida.
- **Distill.pub** · Artículos visuales sobre CNN y backprop.
- **Tensorflow Playground** · playground.tensorflow.org — entrena redes en el navegador.
- **thispersondoesnotexist.com** · refresca y flipa.
- **Karpathy** · «The Unreasonable Effectiveness of RNNs». Lectura obligada.